Insights to Impact Bounty: Analytics Engineer

Palash Pawar · palash.pawar@utexas.edu · Statistics & Data Science @ The University of Texas at Austin

Exercise 1: Design the Data Model

Assumptions: the three tables are `auth_events` (sign-up, email verification; keyed by `account_uuid`), `kyc_events` (identity verification; keyed by email hash), and `ledger_events` (deposits, trades; keyed by numeric `legacy_user_id`). Timestamps arrive as ISO strings with offsets, epoch seconds, and naive local time.

Layers (each problem solved once, in one place):

1. **Staging (one model per source):** cast every timestamp to UTC (naive ones using the documented source timezone), deduplicate on `event_id`, keep `_loaded_at`. Unparseable rows go to quarantine, never silently dropped.
2. **Identity map:** `int_user_identity` maps every source identifier to one canonical `user_id` (`account_uuid`: created at sign-up, never reused). Everything depends on this join, so it gets its own tests.
3. **Marts:** a long `fct_onboarding_events` (every event) and a wide `fct_onboarding_funnel` (**one row per user**) for PMs.

```sql
-- fct_onboarding_funnel  (grain: one row per canonical user_id)
WITH steps AS (
  SELECT user_id,
         MIN(CASE WHEN step = 'signup'         THEN event_ts_utc END) AS signup_at,
         MIN(CASE WHEN step = 'email_verified' THEN event_ts_utc END) AS email_verified_at,
         MIN(CASE WHEN step = 'idv_verified'   THEN event_ts_utc END) AS idv_verified_at,
         MIN(CASE WHEN step = 'deposit'        THEN event_ts_utc END) AS first_deposit_at,
         MIN(CASE WHEN step = 'trade'          THEN event_ts_utc END) AS first_trade_at
  FROM fct_onboarding_events
  GROUP BY user_id
)
SELECT
  user_id,
  CAST(signup_at AS DATE)                                          AS signup_cohort_date,
  signup_at, email_verified_at, idv_verified_at, first_deposit_at, first_trade_at,
  -- "reached" = event exists; "strict" = reached AND every prior step reached first
  first_trade_at IS NOT NULL                                       AS reached_trade,
  (signup_at <= email_verified_at AND email_verified_at <= idv_verified_at
   AND idv_verified_at <= first_deposit_at AND first_deposit_at <= first_trade_at)
                                                                   AS strict_funnel_complete,
  CASE WHEN first_trade_at   IS NOT NULL THEN 'trade'
       WHEN first_deposit_at IS NOT NULL THEN 'deposit'
       WHEN idv_verified_at  IS NOT NULL THEN 'idv'
       WHEN email_verified_at IS NOT NULL THEN 'email'
       ELSE 'signup' END                                           AS furthest_stage,
  signup_at IS NULL                                                AS is_missing_signup,
  NOT COALESCE(strict_funnel_complete, TRUE)                       AS has_out_of_order_steps
FROM steps;
```

Key decisions and tradeoffs

- **Missing events:** NULL means “not reached.” Users with downstream events but **no sign-up** (usually a failed identity match or migrated account) are kept, flagged `is_missing_signup`, and excluded from cohort metrics by default.
- **Out-of-order events:** keep real timestamps and **never re-sequence**; reordering hides upstream bugs (clock skew, backfilled KYC). The flag lets analysts choose “reached” vs. strict conversion; I’d document which the deck uses.
- **Repeats:** `MIN()` takes the *first* occurrence; the long table keeps retries.
- **Late data:** incremental builds reprocess a 7-day lookback so late KYC callbacks land in the right cohort.

Data quality checks (run on every build; failures block publishing):

Check	Why
`user_id` unique and not null in the funnel table	Enforces the grain
Identity-map match rate ≥ 99.5% per source; unmatched IDs listed	Catches broken joins before they become “drop-off”
Row counts reconcile: distinct users per source vs. users in the mart	Nothing is silently lost
No future timestamps; none before the product launched	Catches timezone and epoch-unit bugs
`has_out_of_order_steps` rate within its historical band (alert if > 2×)	Detects upstream regressions
Freshness: newest event is less than 6h old	Stale data looks like a conversion drop

---

Exercise 2: The Data Doesn’t Add Up

Assumption: each extract has `user_id` and `signup_ts` (`event_type`, `is_test` used if present). Tested on synthetic extracts reproducing the formatting problems.

```
import argparse
import pandas as pd

def load(path, name):
    df = pd.read_csv(path, dtype={"user_id": str})
    df["system"], df["raw_user_id"] = name, df["user_id"]
    # Normalize IDs: whitespace, case, legacy prefixes
    df["user_id"] = (df["user_id"].astype(str).str.strip().str.lower()
                     .str.replace(r"^(usr_|u-)", "", regex=True))
    df.loc[df["raw_user_id"].isna(), "user_id"] = pd.NA
    # Mixed timestamp formats -> UTC
    ts = df["signup_ts"].astype(str)
    is_epoch = ts.str.fullmatch(r"\d{10}")
    parsed = pd.to_datetime(ts.where(~is_epoch), utc=True, errors="coerce", format="mixed")
    parsed[is_epoch] = pd.to_datetime(ts[is_epoch].astype("int64"), unit="s", utc=True)
    df["signup_ts_utc"] = parsed
    return df

def profile(df):
    ids = df["user_id"]
    return {"rows": len(df), "distinct_users": ids.nunique(),
            "null_ids": int(ids.isna().sum()),
            "dup_rows_exact": int(df.duplicated(["user_id", "signup_ts"]).sum()),
```

```
            "dup_ids_any": int(ids.dropna().duplicated().sum()),
            "unparseable_ts": int(df["signup_ts_utc"].isna().sum()),
            "test_accounts": int(df["is_test"].fillna(False).astype(bool).sum())
                             if "is_test" in df else "n/a"}

def main():
    p = argparse.ArgumentParser()
    p.add_argument("a"); p.add_argument("b"); p.add_argument("c")
    p.add_argument("--date", required=True); p.add_argument("--tz", default="UTC")
    args = p.parse_args()
    systems = {n: load(f, n) for n, f in zip("ABC", [args.a, args.b, args.c])}

    print("== 1. Profile ==")
    print(pd.DataFrame({n: profile(d) for n, d in systems.items()}).to_string())

    print("\n== 2. System C duplicates: re-ingest or multiple events? ==")
    c = systems["C"]
    dups = c[c["user_id"].duplicated(keep=False) & c["user_id"].notna()]
    if len(dups):
        g = dups.groupby("user_id")["signup_ts_utc"].nunique()
        print("same ts (re-ingest):", int((g == 1).sum()),
              "| different ts (retries/multi-event):", int((g > 1).sum()))
        if "event_type" in c:
            print(dups["event_type"].value_counts().to_string())

    print("\n== 3. Day-boundary sensitivity ==")
    day = pd.Timestamp(args.date)
    for tz in ["UTC", "America/Los_Angeles", "America/New_York"]:
        start = day.tz_localize(tz).tz_convert("UTC"); end = start + pd.Timedelta(days=1)
        print(f"{tz:22s}", {n: d.loc[d.signup_ts_utc.between(start, end, inclusive="left"),
                                     "user_id"].nunique() for n, d in systems.items()})

    print(f"\n== 4. Overlap of deduped IDs ({args.tz} day) ==")
    start = day.tz_localize(args.tz).tz_convert("UTC"); end = start + pd.Timedelta(days=1)
    sets = {n: set(d.loc[d.signup_ts_utc.between(start, end, inclusive="left"),
                         "user_id"].dropna()) for n, d in systems.items()}
    for x in "ABC":
        others = set().union(*(sets[y] for y in "ABC" if y != x))
        print(f"only in {x}: {len(sets[x] - others)} | missing from {x}: {len(others -
        sets[x])}")
    allids = set().union(*sets.values())
    pd.DataFrame([{"user_id": u, **{n: u in s for n, s in sets.items()}} for u in allids]) \
      .to_csv("signup_presence_matrix.csv", index=False)

if __name__ == "__main__":
    main()
```

How I'd investigate: separate *definition* differences from *data* differences:

1. **C's duplicates (57K):** are they re-ingestion (identical timestamps: pipeline bug) or multiple events per user (`started` + `completed`: C counts events, not people)? If deduplicating brings C near A, that's most of the gap.
2. **Day boundaries:** systems may cut "the day" in UTC, PT, or ET. Section 3 re-counts all three on one boundary.

3. **What counts as a signup:** I'd join users missing from B to the users table and look for a pattern. Hypotheses for B (45K): counts only *verified* signups, excludes internal/test accounts, or pipeline lag. Each has a signature: missing users are all unverified, all internal, or all from the day's last hours.
4. **Owners:** one question to each: "What event and timezone define a signup in your system?"

What I'd tell the PM (Wednesday, not Friday morning):

> "The three numbers measure different things, not the same thing wrong. C counts signup *events* (users appear more than once); after deduplication it's ~X. B appears to count only verified signups. I recommend **A's number, [52K] unique users who completed sign-up (UTC day)**, with a footnote stating the definition. Confidence: high on the direction, ±[N]% on the level until I confirm B's filter with its owner. I'll update you Thursday either way. If we can't close the gap by then, I'll give you a range and one sentence to use if leadership asks."

I'd flag C's duplication (it likely inflates other metrics built on C) and the missing shared signup definition in a ticket, not the deck.

Exercise 3: The Inherited Pipeline

What I'd flag, in order of impact:

1. **The join to `devices` multiplies rows.** Each trade repeats once per active device. `COUNT(DISTINCT)` survives, but `completed_trades` is **inflated for any user with 2+ devices**, likely the double-count behind Exercise 4.
2. **`d.is_active` is current state, not state as of the week.** Users who later deactivate devices vanish from past weeks; history changes on every re-run.
3. **The device filter defines the metric.** Traders with no active device (e.g., API users) are excluded. Intended or not, it's undocumented.
4. **Status codes 1 and 3 are undocumented.** Does "completed" include partial fills?
5. **`u.country` is current country.** Movers are restated into their new country.
6. **Comma joins and hard-coded dates.** Implicit joins hide accidental cross joins; the window has no timezone (the half-open range is correct).

Getting the country cut done without breaking the metric:

- **Freeze a baseline first.** Save the current pipeline's output for 12 weeks as my regression test.
- **Don't touch the production logic this week.** The query already groups by country, so I'd expose a **new model reusing the existing logic as-is**, so it uses the same definition as the published total. One test: country rows must sum exactly to the published total (one country per user). Nulls get an explicit `'unknown'` bucket.
- **Deliver with caveats.** "Country = current country on the user profile; counts follow the existing metric definition, including its device filter."
- **Fix the real problems separately.** Ticket the fan-out, device state and status codes; each fix ships separately with a before/after diff and metric-owner sign-off.

Using AI: I'd have an AI assistant produce column-level lineage for all 400 lines, list every join's expected cardinality, and draft dbt tests, all treated as hypotheses. **Verification:** (1) any refactored version must reproduce the frozen baseline exactly for all 12 weeks, row by row; (2) I'd hand-trace 5 to 10 users, including

one with multiple devices and one with no device, through both versions; (3) I'd check each join's cardinality directly with `COUNT(*)` vs `COUNT(DISTINCT key)`. Unconfirmed explanations aren't used.

Exercise 4: Your Number Is "Wrong"

What I'd say to the PM, in private, in under a minute:

> "I can't make my table produce 41%. It would mean building a known double-count into the new model, and everyone who uses it later would inherit that. But you don't need to show a discrepancy today. Present the dashboard number as it is. It's the series leadership has tracked for two years, and the trend on it is still meaningful. If someone asks, the honest line is: *'We're moving to a deduplicated definition that counts each person once. It comes in lower, around 34%, and we'll bring the restated history next week.'* I'll stand behind that with you."

This protects the PM in the room, keeps a known-wrong number out of the new source of truth, and makes the correction the PM's upgrade, not my "gotcha."

Moving to one trusted definition (the next two weeks):

1. **Quantify the impact, not just the error.** Restate both definitions over two years. Is the 7-point gap **stable**? If so, trends and most past decisions hold; only the level shifts. That's the most reassuring message for leadership. If it *grew* (as multi-device use grew), I'd identify which decisions relied on that trend.
2. **Write a one-page metric definition:** numerator, denominator, deduplication rule, status codes, timezone, and an owner. Get it signed off by the Growth PM and the analytics lead.
3. **Run both in parallel, clearly labeled** ("WAT trade rate (legacy)" vs. "(v2, deduplicated)"), for 2 to 4 weeks, with a note on the legacy dashboard explaining the difference.
4. **Deprecate the legacy metric on a set date** that the PM announces.
5. **Prevent a repeat:** add a uniqueness test on the metric's grain, so a fan-out join fails the build instead of flowing into a deck.

The goal isn't winning 34% vs 41%; it's one owned definition everyone understands.

AI Disclosure

I used Claude (Anthropic) as a drafting and coding assistant: it helped structure and draft my responses and wrote the initial Exercise 2 script, which I tested against synthetic data. I directed the approach, edited the final text, and can defend every answer and line of code without AI assistance.